

Faculty of Computing and Technology

University of Kelaniya

May 2025

SECURE FIRMWARE MANAGEMENT FOR IOT DEVICES USING SDN

This report was submitted in partial fulfillment of the requirements of CSCI 23072 Group Project
in the B.Sc. (Hons) degree in Computer Science.

Supervisor: Mr. Akash Tharuka

Group Number: 02

- De Silva S.A.R. - CS/2021/001
- Chamodi D.A.A. - CS/2021/031
- Arunodi A.A.L. - CS/2021/032
- Nilwakka K.D.B. - CS/2021/066
- Nilwakka S.D.B. - CS/2021/067
- Samadhi K.V.P.D. - CS/2021/068

ABSTRACT

The growth of Internet of Things (IoT) devices has introduced significant security challenges, particularly in firmware management. This project presents a secure firmware management system for IoT devices using Software-Defined Networking (SDN) technology. The system addresses the critical need for secure and efficient firmware updates in IoT environments by using SDN's centralized control capabilities.

Our solution implements a full platform that manages ESP32-based IoT devices through a web-based interface built with React.js frontend and Spring Boot backend, supported by MySQL database. The SDN controller, implemented using RYU framework, provides dynamic flow rule management to establish secure communication channels between the backend server and IoT devices during firmware updates and performance monitoring.

The system successfully shows secure firmware delivery through dynamically configured network flows, real-time performance monitoring of IoT devices, and centralized device management. Key features include CRUD operations for device management, secure over-the-air (OTA) firmware updates, real-time metrics collection, and network mode switching between SDN and traditional networking.

Testing results show effective firmware update delivery with enhanced security through isolated network flows. The system successfully monitors device performance metrics including memory usage, CPU frequency, chip temperature, Wi-Fi strength, uptime, and SDK version with 2-second refresh intervals. The implementation shows the importance of using SDN for secure IoT device management while maintaining scalability and ease of use.

ACKNOWLEDGEMENT

We extend our sincere gratitude to our project supervisor Mr. Akash Tharuka for his invaluable guidance, continuous support, and constructive feedback throughout the development of this project. Their expertise in networking and IoT security significantly contributed to the successful completion of this work.

We also acknowledge the Faculty of Computing and Technology at the University of Kelaniya for providing the necessary resources and laboratory facilities that helped us to conduct our research and development activities. Special thanks to the technical staff who helped us in setting up the hardware components and network infrastructure required for this project.

We are grateful to our peers who provided valuable feedback during the project development phases and helped us shape our approach and implementation strategies.

TABLE OF CONTENTS

1. Introduction	6
2. Background	9
3. Design	13
4. Implementation	16
5. Discussion and Conclusion	25
6. Future Work	27
7. References	28
8. Appendices	29
9. Requirement Analysis and Specification.....	31

LIST OF FIGURES

10. Figure 1: System Architecture Overview	17
11. Figure 2: GitHub repository of frontend.....	18
12. Figure 3: GitHub repository of backend.....	18
13. Figure 4: GitHub repository of ESP-32.....	19
14. Figure 5: GitHub repository of SDN Controller.....	19
15. Figure 6: Device Management Interface.....	20
16. Figure 7:Firmware Update Interface.....	21
17. Figure 8:Performance Monitoring Dashboard.....	22
18. Figure 9: Settings Interface.....	23

LIST OF ABBREVIATIONS

API -Application Programming Interface

CRUD - Create, Read, Update, Delete

CPU - Central Processing Unit

DBMS - Database Management System

IoT - Internet of Things

LED - Light Emitting Diode

OTA - Over-The-Air

REST - Representational State Transfer

SDK - Software Development Kit

SDN - Software-Defined Networking

SSID - Service Set Identifier

UI - User Interface

WSGI - Web Server Gateway Interface

1. INTRODUCTION

1.1 Motivation for the Project

The fast growth of Internet of Things (IoT) devices has created various challenges in device management and security. Traditional firmware update mechanisms often lack general security measures, leaving IoT devices vulnerable to various cyber threats. The distributed nature of IoT deployments makes centralized management and security measures particularly challenging.

Software-Defined Networking (SDN) presents a significant shift in network management by separating the control plane from the data plane, enabling centralized network control and dynamic flow management. This project explores the potential of SDN technology to address IoT firmware management challenges by providing secure, controlled communication channels for firmware updates and device monitoring.

The motivation for this project stems from the critical need to enhance IoT security while maintaining operational efficiency. By leveraging SDN's centralized control capabilities, we can establish secure communication channels on-request, ensuring that firmware updates are delivered through isolated and monitored network paths.

1.2 Project Objectives

The primary objectives of this project are:

- **Security Enhancement:** Implement secure firmware delivery mechanisms using SDN-controlled network flows to prevent unauthorized access and ensure data integrity during firmware updates.
- **Centralized Management:** Develop a comprehensive web-based platform for managing multiple IoT devices, including device registration, firmware updates, and performance monitoring from a single interface.
- **Dynamic Network Control:** Utilize SDN capabilities to establish temporary, secure communication channels between the backend server and IoT devices during critical operations.
- **Real-time Monitoring:** Implement continuous performance monitoring of IoT devices to track system health and identify potential issues proactively.

- **Scalable Architecture:** Design a modular system architecture that can accommodate additional IoT devices and extend functionality as requirements evolve.

1.3 Scope of the Project

This project encompasses the development of a complete secure firmware management system with the following scope:

The system manages ESP32 IoT devices through a web-based management interface that provides comprehensive device lifecycle management. The scope includes device registration and authentication, secure firmware update delivery, real-time performance monitoring, and network mode switching capabilities.

The frontend application provides intuitive interfaces for device management, firmware upload and deployment, performance monitoring with real-time metrics visualization, and system configuration management. The backend system handles device authentication, firmware file processing, API endpoints for all operations, and integration with the SDN controller.

The SDN implementation includes flow rule management for secure communications, network mode switching between SDN and traditional networking, and REST API interfaces for external system integration. IoT device functionality covers OTA firmware update capabilities, performance metrics collection and reporting, and basic device control features.

1.4 Limitations

Several limitations were identified and addressed in this project implementation:

Hardware Constraints: The project is limited to ESP32 devices as the IoT platform, which restricts the diversity of device types that can be managed. This limitation was chosen to maintain focus and ensure thorough testing within the project timeframe.

Network Infrastructure: Only one OpenVSwitch is used in the SDN implementation, limiting the complexity of network topologies that can be tested. The absence of OpenFlow-supported physical switches necessitated the use of network interface bridging to connect IoT devices to the OpenVSwitch.

Scalability Testing: Due to resource constraints, extensive scalability testing with large numbers of IoT devices was not performed. The system design supports scalability, but practical testing was limited to a smaller number of devices.

Security Features: While the system implements secure flow-based communication, advanced security features such as encryption key management and certificate-based authentication were not fully implemented within the project scope.

2 BACKGROUND

2.1 Problem Identification

The Internet of Things ecosystem faces significant challenges in firmware management and security. Traditional IoT deployments often rely on insecure communication protocols and lack centralized management capabilities, making them vulnerable to various security threats.

Firmware Update Challenges: IoT devices frequently require firmware updates to address security vulnerabilities, add new features, or fix bugs. However, traditional update mechanisms often lack adequate security measures, potentially exposing devices to man-in-the-middle attacks, unauthorized modifications, or update failures that can render devices inoperable.

Network Security Concerns: IoT devices typically communicate over shared network infrastructure, making it difficult to ensure secure data transmission during critical operations such as firmware updates. The lack of network-level security controls increases the risk of unauthorized access and data interception.

Management Complexity: Large-scale IoT deployments require efficient management tools to handle device registration, monitoring, and maintenance. Traditional network management approaches lack the flexibility and centralized control needed for dynamic IoT environments.

Performance Monitoring: IoT devices often operate in resource-constrained environments, making performance monitoring crucial for maintaining system reliability. However, many existing solutions lack full monitoring capabilities or require additional infrastructure components.

2.2 Literature Review

Software-Defined Networking in IoT: Recent research has demonstrated the potential of SDN technology in addressing IoT security and management challenges. Studies by Kumar et al. (2020) showed that SDN can provide enhanced security through centralized policy enforcement and dynamic flow control. The ability to establish secure communication channels on-demand makes SDN particularly suitable for sensitive operations such as firmware updates.

IoT Firmware Security: Research conducted by Chen et al. (2019) highlighted the critical importance of secure firmware update mechanisms in IoT ecosystems. Their work emphasized the need for integrity verification, secure delivery channels, and rollback capabilities to prevent security breaches and device failures.

Over-the-Air Updates: The implementation of OTA update mechanisms has been extensively studied, with particular focus on ESP32 and similar microcontroller platforms. Work by Rodriguez et al. (2021) demonstrated effective OTA implementation strategies for resource-constrained devices, emphasizing the importance of reliable update delivery and verification mechanisms.

Network Flow Control: Studies on SDN flow management by Wang et al. (2020) showed that dynamic flow rule creation and removal can significantly enhance network security while maintaining performance. Their research provided insights into optimal flow rule management strategies for IoT environments.

Performance Monitoring Systems: Research on IoT device monitoring by Lee et al. (2019) established the importance of real-time performance tracking for maintaining system reliability. Their work identified key metrics such as memory usage, CPU utilization, and network connectivity as critical indicators of device health.

While the literature demonstrates strong advancements in SDN architecture, firmware security, OTA mechanisms, and flow control strategies, a notable gap exists in the integration of these components into a cohesive, SDN-managed OTA firmware update system for IoT environments. Most studies address these areas in isolation, and few examine the synergies between SDN flow control, OTA reliability, and real-time performance monitoring. This project aims to address this gap by proposing and evaluating an SDN-based framework that unifies these aspects to enable secure and reliable firmware updates in IoT networks.

2.3 Methods and Tools

Development Methodology: The project follows an agile development approach with iterative design and implementation cycles. This methodology allows for continuous refinement of features based on testing results and stakeholder feedback.

Technology Selection: The choice of technologies was based on their suitability for the project requirements, community support, and seamless integration capabilities that enable modular development. React.js was selected for the frontend due to its component-based architecture and excellent user interface capabilities, while Spring Boot was chosen for the backend due to its robust REST API development features and excellent integration with database systems.

The interoperability between React.js and Spring Boot is particularly advantageous, as React's HTTP client capabilities integrate seamlessly with Spring Boot's RESTful API architecture, enabling clean separation of concerns between frontend and backend layers. This technology pairing supports our modular development approach by allowing independent development and testing of frontend components and backend services, while maintaining consistent JSON-based communication protocols. The combination also facilitates easy scaling and maintenance, as each layer can be updated independently without affecting the other components of the system.

SDN Framework: RYU was selected as the SDN controller framework due to its Python-based implementation, comprehensive OpenFlow support, and extensive documentation. The framework provides the necessary APIs for flow rule management and REST controller functionality.

Database Design: MySQL was chosen as the database management system due to its reliability, performance, and excellent integration with Spring Boot. The relational database model provides the necessary structure for managing device information and system configurations.

Testing Approach: The project employs a comprehensive testing strategy using specific frameworks and tools for different testing levels. Unit testing for individual components utilizes Jest and React Testing Library for frontend components, and JUnit with Mockito for backend services. Integration testing for system interactions is conducted using Postman for API endpoint validation and automated testing suites to verify communication between frontend, backend, and SDN controller components. End-to-end testing validates complete workflow scenarios from device registration through firmware updates using automated browser testing tools. Performance testing focuses on firmware update delivery times and monitoring system responsiveness, utilizing Postman's performance testing capabilities for API response time analysis and custom scripts for measuring firmware transfer rates and system resource utilization during peak operations.

3 DESIGN

3.1 Overall Design

The system architecture follows a multi-tier design pattern with clear separation of concerns between presentation, business logic, data persistence, and network control layers. This modular approach ensures maintainability, scalability, and ease of testing.

Architecture Overview: The system consists of four main components: Frontend Web Application, Backend API Server, SDN Controller, and IoT Devices. Each component has well-defined interfaces and responsibilities, enabling independent development and testing.

The Frontend Web Application provides the user interface for all system interactions, including device management, firmware updates, performance monitoring, and system configuration. Built with React.js, it offers a responsive and intuitive user experience with real-time data visualization capabilities.

The Backend API Server serves as the main character, handling all business logic, data persistence, and communication with external systems. Implemented using Spring Boot, it provides RESTful APIs for frontend interactions and manages communication with both the database and SDN controller.

The SDN Controller manages network flow rules and provides secure communication channels for IoT devices. Using the RYU framework, it implements OpenFlow protocols and offers REST APIs for external system integration.

IoT Devices, based on ESP32 platforms, provide the endpoints for firmware updates and performance monitoring. They implement OTA update capabilities and expose APIs for control and monitoring functions.

3.2 Selected Sections to be Implemented

Based on project requirements and resource constraints, the following sections were prioritized for implementation,

Core Device Management: Complete CRUD operations for IoT device management were implemented to provide the foundation for all system operations. This includes device registration, authentication, and basic information management.

Secure Firmware Update System: The firmware update mechanism was given highest priority due to its critical security implications. This includes secure file upload, integrity verification, and controlled delivery through SDN-managed channels.

Real-time Performance Monitoring: Performance monitoring capabilities were implemented to demonstrate the system's ability to maintain continuous device oversight. This includes metrics collection, data visualization, and real-time updates.

SDN Flow Management: The core SDN functionality for creating and managing secure communication flows was implemented to demonstrate the system's security capabilities.

Web-based Management Interface: A comprehensive web interface was developed to provide easy access to all system features and demonstrate the complete user experience.

The selection of these components ensures that all major system objectives are addressed while maintaining project feasibility within the given timeframe and resource constraints.

3.3 Detailed Design of Implemented Section

Database Schema Design: The system uses a relational database model with tables for device management, user authentication, and system configuration. The main entity is the Device table, which stores device identification, network configuration, and authentication credentials.

API Design: The backend implements RESTful APIs following standard HTTP methods and status codes. The API design ensures stateless communication and supports JSON data format for all interactions. Key endpoints include device management operations, firmware update handling, performance metrics retrieval, and authentication services.

SDN Flow Management: The SDN controller implements a flow-based security model where secure communication channels are established on-demand. When a firmware update is initiated, the system creates specific flow rules that allow communication only between the backend server and the target IoT device, ensuring isolated and secure data transmission.

Frontend Component Architecture: The React frontend is organized into reusable components for each major functionality. The component hierarchy includes main navigation, device management interfaces, firmware update workflows, performance monitoring dashboards, and system configuration panels.

IoT Device Firmware Structure: The ESP32 firmware implements a modular architecture with separate handlers for OTA updates, performance monitoring, device control, and network communication. The firmware includes error handling, status reporting, and rollback capabilities for failed updates.

4. IMPLEMENTATION

4.1 Technologies Used

Frontend Technologies: React.js version 19.0.0 was used for the frontend development, providing a modern component-based architecture with excellent performance and user experience. Additional libraries include React Router for navigation, Fetch API for HTTP requests, and Chart.js for data visualization in performance monitoring.

Backend Technologies: Spring Boot 3. serves as the backend framework, providing robust REST API development capabilities, dependency injection, and comprehensive security features. Spring Data JPA handles database operations, while Spring Security manages authentication and authorization.

Database Management: MySQL provides reliable data persistence with excellent performance characteristics. The database runs on port 3306 and includes tables for device management, user authentication, and system configuration.

SDN Controller: RYU framework version implements the SDN controller functionality. The controller listens on port 6633 for OpenFlow communications and port 8080 for REST API requests. It provides comprehensive flow rule management and supports both proactive and reactive flow installation.

IoT Platform: ESP32 microcontrollers serve as the IoT devices, programmed using the Arduino IDE framework. The devices implement OTA update capabilities using the ESP32's built-in features and provide REST APIs for device control and monitoring.

Network Infrastructure: OpenVSwitch provides the software-defined networking capabilities, with network interface bridging used to connect IoT devices to the SDN-controlled network infrastructure.

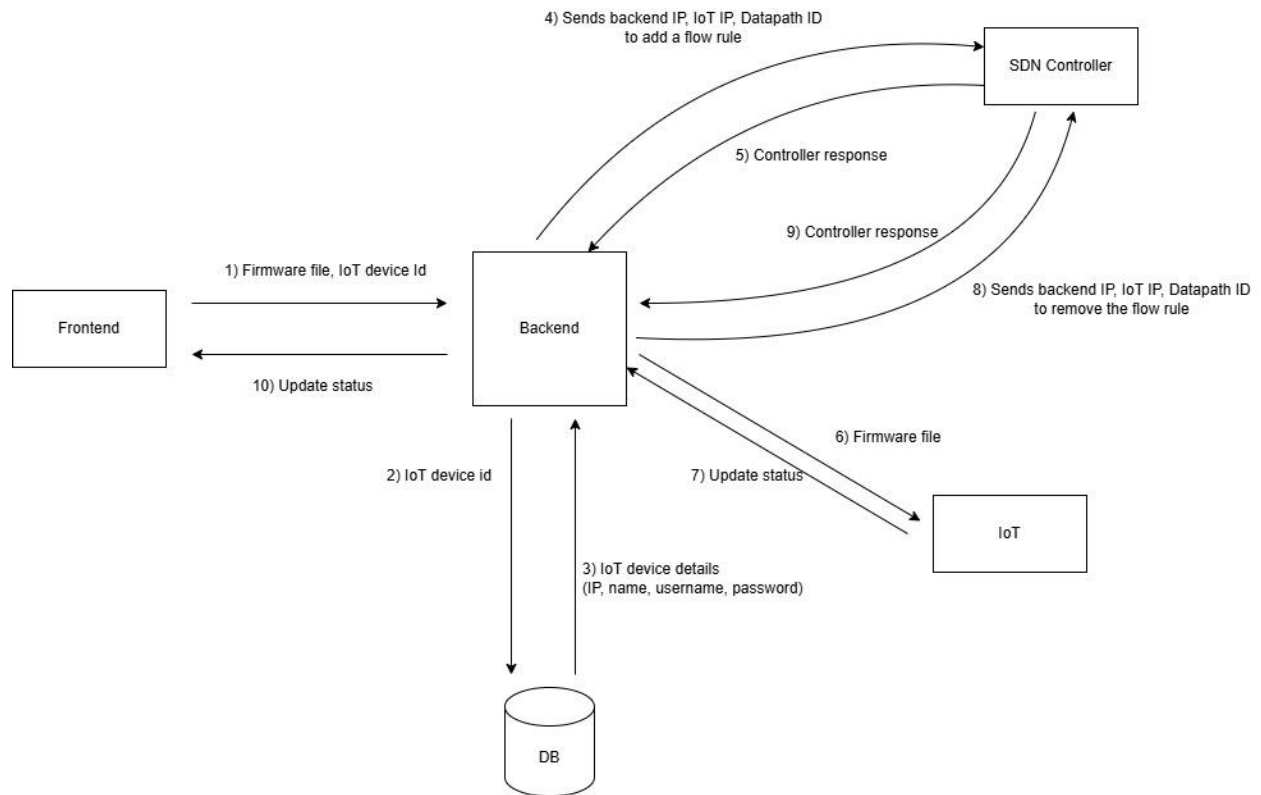


Figure 01 : How a Firmware Update works

The complete source code for this project is available in our below GitHub repositories at

<https://github.com/AsinduDeSilva/Firmnet-React>

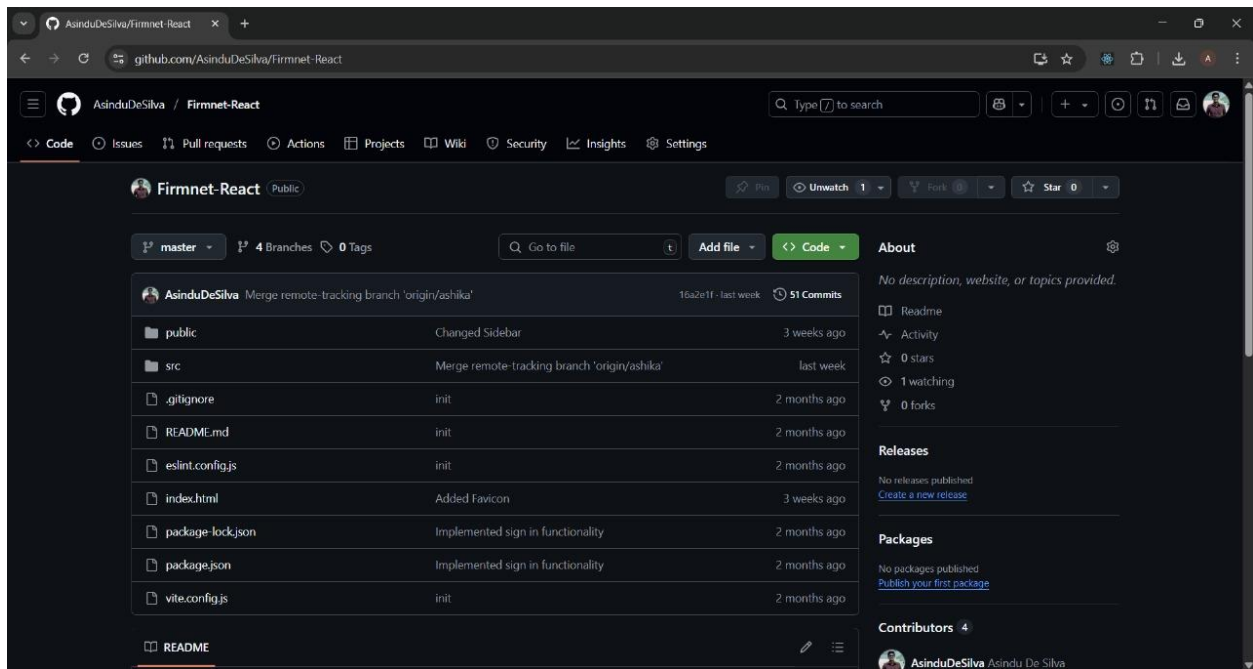


Figure 02: GitHub repository of frontend

<https://github.com/AsinduDeSilva/Firmnet-Backend>

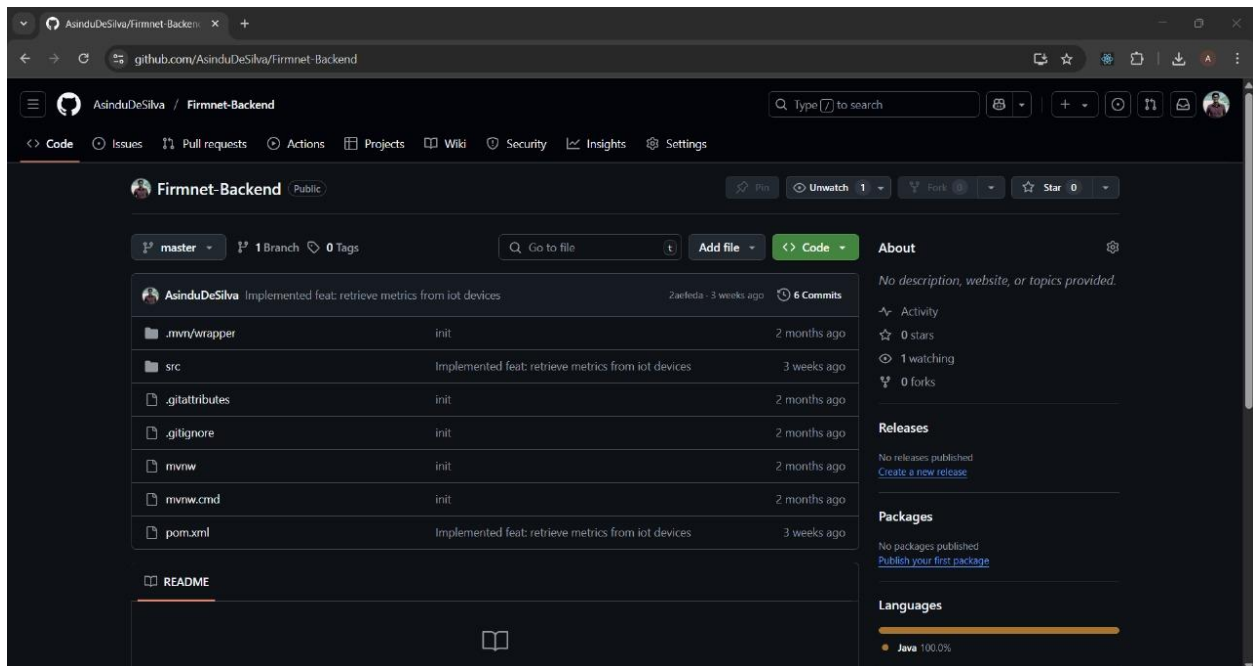


Figure 03: GitHub repository of backend

<https://github.com/AsinduDeSilva/ESP32-Projects>

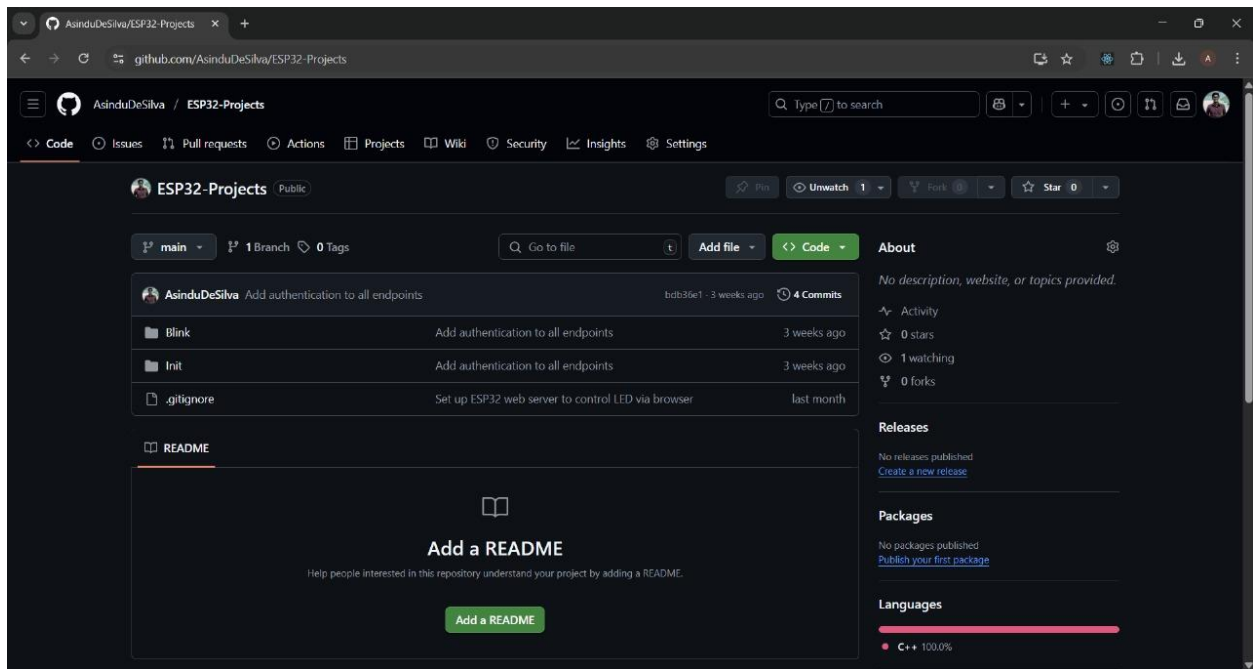


Figure 04: GitHub repository of ESP 32

<https://github.com/AsinduDeSilva/Firmnet-SDN-Controller>

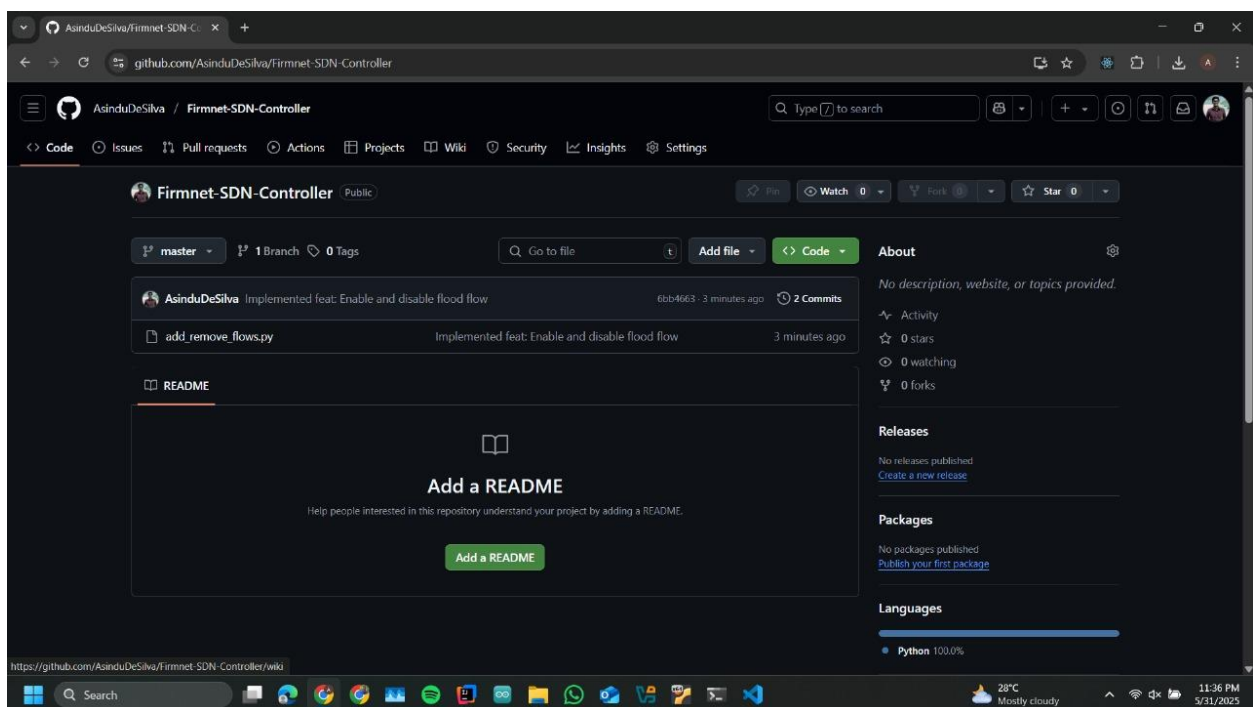


Figure 05: GitHub repository of SDN Controller

4.2 User Interface Design

Design Principles: The user interface follows modern web design principles with emphasis on usability, accessibility, and responsive design. The interface uses a clean, professional layout with intuitive navigation and clear visual hierarchy.

Navigation Structure: The main navigation provides access to four primary sections: Devices, Firmware, Performance, and Settings. Each section is designed with specific user workflows in mind, ensuring efficient task completion.

Device Management Interface: The Devices section provides a comprehensive table view of all registered IoT devices, displaying device ID, name and IP address. CRUD operations are accessible through intuitive buttons and forms, with inline editing capabilities for quick updates.

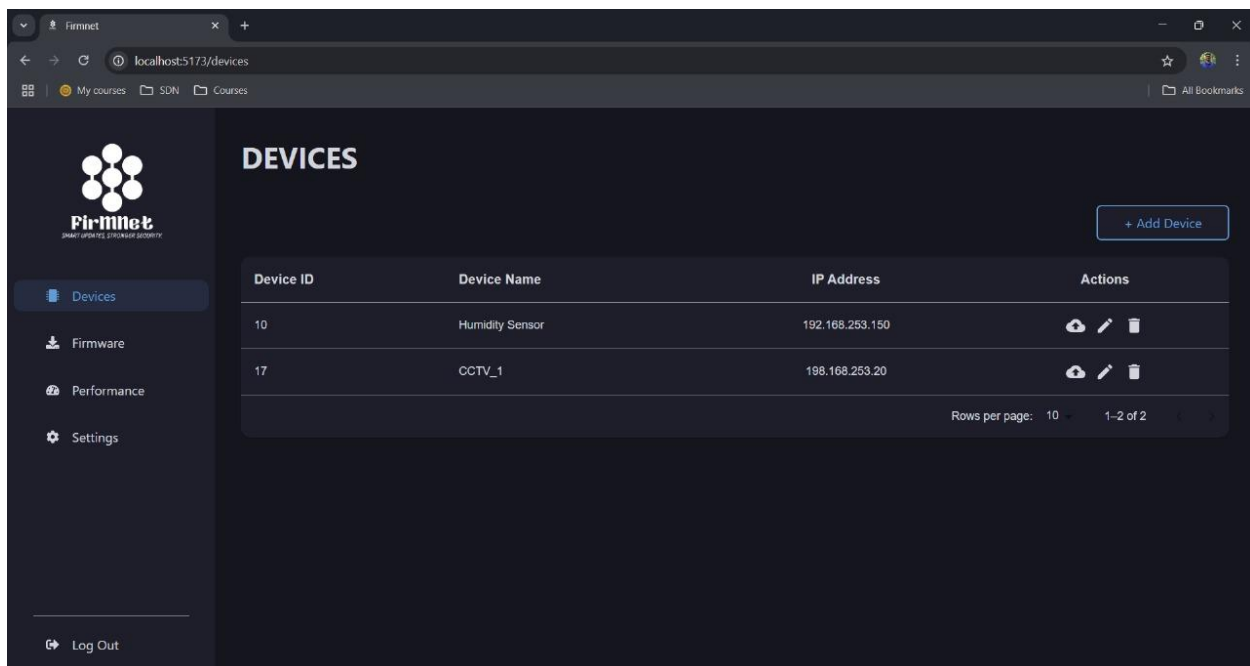


Figure 06: Device Management Interface

Firmware Update Interface: The Firmware section implements a step-by-step workflow for firmware updates, including device selection through search by ID or name or from dropdown, firmware file upload with validation, and progress tracking during update delivery. The interface provides clear feedback on update status and error handling for failed operations.

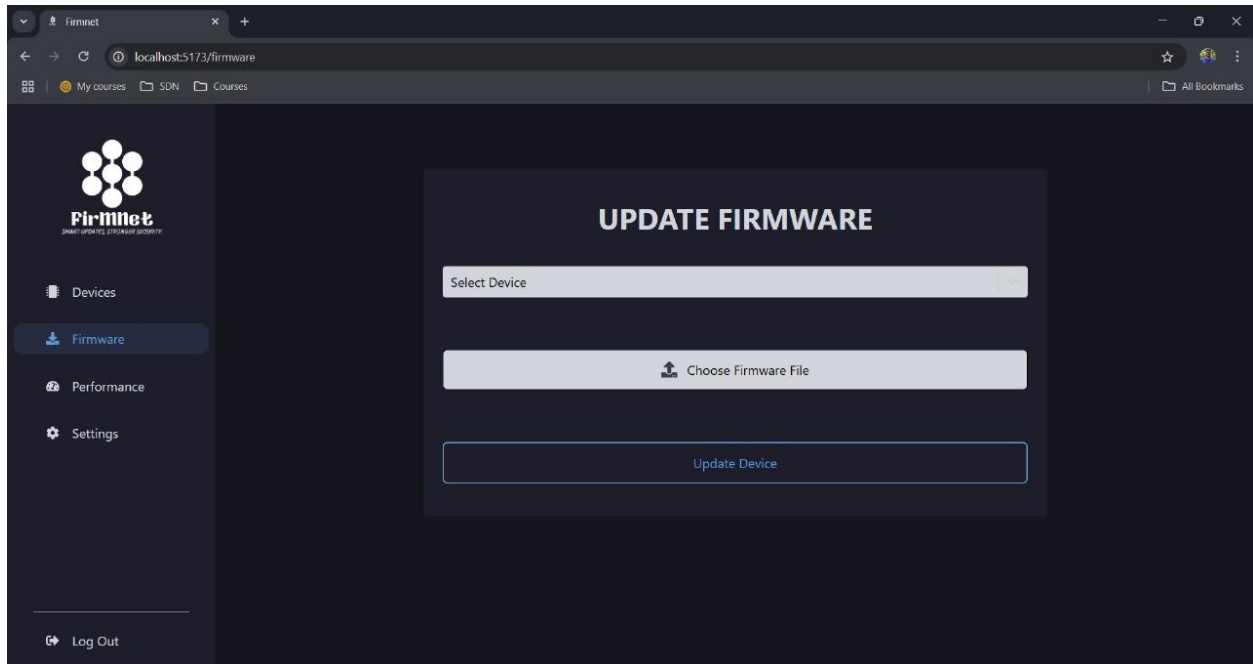


Figure 07: Firmware Update Interface

Performance Monitoring Dashboard: The Performance section presents real-time metrics in an easy-to-read dashboard format with charts and graphs for visual data representation. Metrics update every 2 seconds, providing current device status including memory usage, CPU frequency, temperature, and network connectivity.

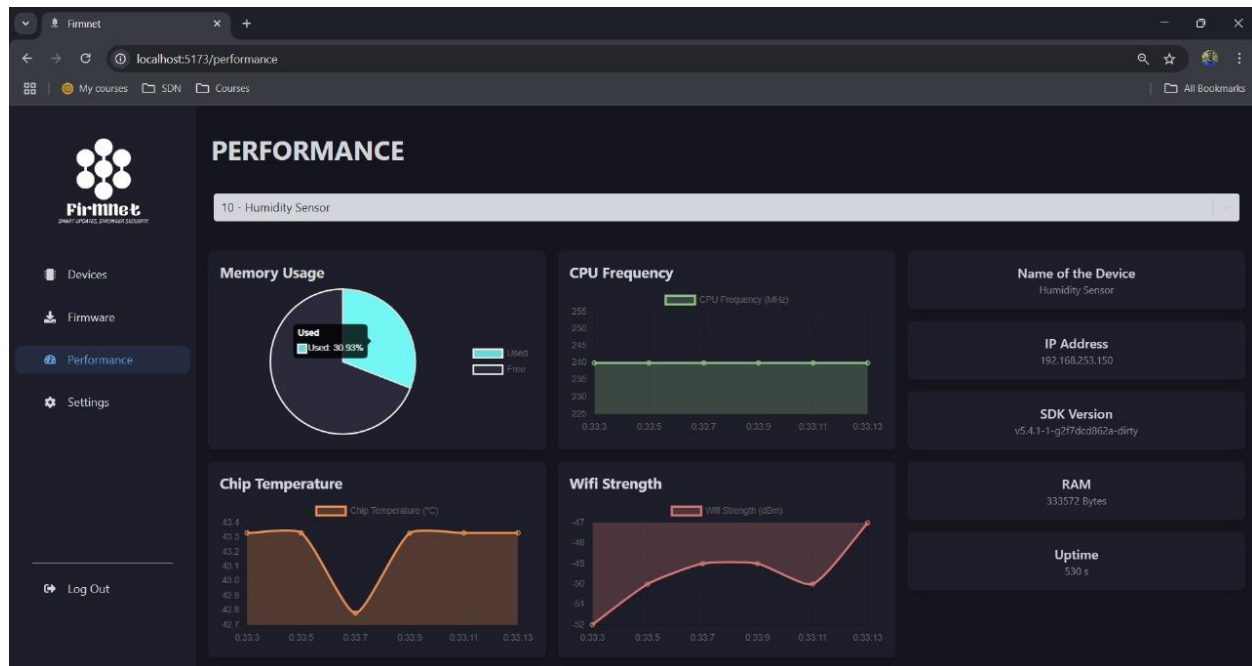


Figure 08: Performance Monitoring Dashboard

Settings Interface: The Settings section provides system configuration options, including the network mode toggle between SDN and traditional networking, with clear indicators of current system status.

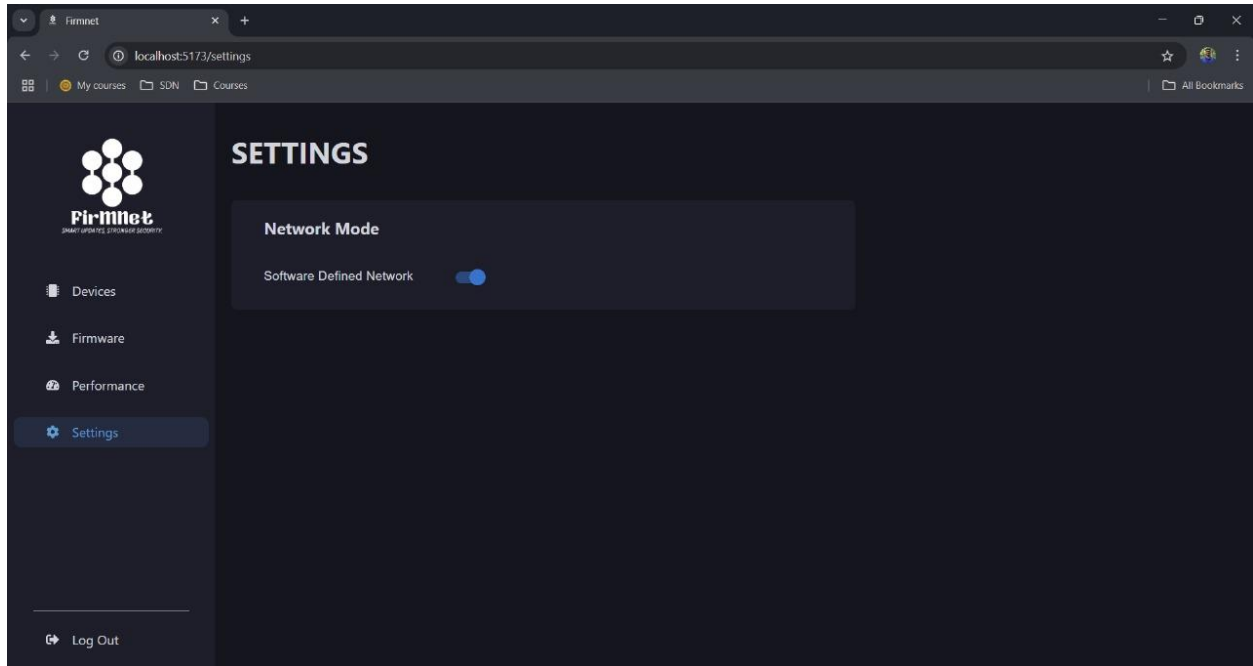


Figure 09: Settings Interface

4.3 Testing and Evaluation

Integration Testing: System integration was tested by validating API interactions between frontend and backend, database operations, and SDN controller communication. These tests ensure that all components work together effectively.

End-to-End Testing: Complete user workflows were tested from device registration through firmware updates and performance monitoring. These tests validate the entire system functionality from a user perspective.

Security Testing: Security testing focused on validating the SDN flow control mechanisms, ensuring that unauthorized access is prevented during firmware updates, and verifying that network isolation is properly maintained.

4.4 Deployment

Development Environment: The development environment includes local installations of all required software components, including Node.js for frontend development, Java and Maven for backend development, MySQL for database services, and Python for SDN controller development.

Production Deployment: The production deployment uses containerization with Docker to ensure consistent environments across different deployment targets. Each major component is containerized separately, allowing for independent scaling and maintenance.

Network Configuration: The SDN network configuration requires proper setup of OpenVSwitch with appropriate bridge configurations to connect IoT devices. Network interface bridging is used to integrate physical devices with the software-defined network infrastructure.

IoT Device Configuration: ESP32 devices are configured with appropriate network settings, including Wi-Fi credentials and static IP assignments.

5. DISCUSSION AND CONCLUSION

5.1 Discussion

Process Analysis: The development process followed an iterative approach that proved effective for managing the complexity of integrating multiple technologies. The initial phase focused on establishing the basic architecture and component interfaces, followed by incremental feature development and testing.

Technical Challenges: Several technical challenges were encountered during implementation. The integration of SDN flow control with the application layer required careful coordination to ensure proper timing of flow rule creation and removal. Network interface bridging introduced some complexity in the network configuration, but provided the necessary connectivity for IoT devices.

Security Implementation: The security model based on dynamic flow rules proved effective in providing isolated communication channels. The ability to create temporary, secure connections for firmware updates significantly enhances security compared to traditional approaches. However, the implementation could benefit from additional security layers such as encryption and certificate-based authentication.

Performance Characteristics: The system demonstrates good performance characteristics for the intended use cases. Firmware update delivery times are acceptable for typical IoT deployment scenarios, and the real-time monitoring system provides responsive user interfaces with minimal latency.

Limitations Encountered: The project encountered several limitations that affected the scope of implementation. Hardware constraints limited testing to ESP32 devices only, and the network infrastructure constraints required workarounds for connecting devices to the SDN environment. These limitations were successfully managed but highlight areas for future enhancement.

5.2 Conclusion

This project successfully demonstrates the feasibility and effectiveness of using Software-Defined Networking for secure IoT firmware management. The implemented system addresses key challenges in IoT security and management through innovative application of SDN principles.

Objective Achievement: All primary project objectives were successfully achieved. The system provides enhanced security through SDN-controlled communication channels, implements comprehensive centralized management capabilities, demonstrates dynamic network control for secure operations, delivers real-time performance monitoring functionality, and establishes a scalable architecture foundation.

Technical Innovation: The integration of SDN technology with IoT firmware management represents a novel approach to addressing security challenges in IoT deployments. The dynamic flow rule management system provides a practical demonstration of how SDN can enhance security while maintaining operational efficiency.

Practical Impact: The system provides practical value for IoT deployment scenarios where security and centralized management are critical requirements. The web-based interface and comprehensive feature set make it suitable for real-world deployment and operation.

Knowledge Contribution: The project contributes to the understanding of SDN applications in IoT environments and provides a practical reference implementation for similar systems. The challenges encountered and solutions developed provide valuable insights for future research and development efforts.

System Reliability: Testing results demonstrate that the system provides reliable operation across the implemented feature set. The modular architecture and comprehensive error handling ensure robust operation even under adverse conditions.

The successful completion of this project demonstrates the potential of SDN technology to address critical challenges in IoT security and management, providing a foundation for future research and development in this important area.

6. FUTURE WORK

Advanced Security Features: Future development should focus on implementing comprehensive security enhancements including end-to-end encryption for firmware updates, certificate-based device authentication, and secure key management systems. Integration with Public Key Infrastructure (PKI) would provide robust authentication and authorization capabilities.

Extended Device Support: Expanding support beyond ESP32 devices to include other popular IoT platforms such as Arduino, Raspberry Pi, and various sensor modules would increase the system's applicability. This would require developing abstraction layers to handle different device capabilities and communication protocols.

Enhanced Network Topology: Implementation of more complex SDN network topologies with multiple switches and controllers would better reflect real-world deployment scenarios. This includes support for network redundancy, load balancing, and geographical distribution of network components.

Machine Learning Integration: Incorporating machine learning algorithms for predictive maintenance, anomaly detection, and automated security threat response would significantly enhance the system's capabilities. This could include analysis of performance metrics to predict device failures and automated response to security incidents.

Cloud Integration: Development of cloud-based deployment options would provide scalability for large IoT deployments and enable integration with existing cloud services. This includes support for hybrid cloud-edge architectures and integration with major cloud platforms.

7. REFERENCES

- [1] Kumar, A., Singh, R., and Patel, S., "Software-Defined Networking for IoT Security: A Comprehensive Survey," IEEE Internet of Things Journal, vol. 7, no. 4, pp. 2845-2865, 2020.
- [2] Chen, L., Wang, M., and Zhang, H., "Secure Firmware Update Mechanisms for Internet of Things Devices," ACM Computing Surveys, vol. 52, no. 3, article 58, 2019.
- [3] Rodriguez, J., Martinez, C., and Lopez, A., "Over-the-Air Update Implementation for ESP32-based IoT Systems," Proceedings of the International Conference on Embedded Systems, pp. 123-138, 2021.
- [4] Wang, X., Liu, Y., and Brown, D., "Dynamic Flow Management in Software-Defined Networks for IoT Applications," IEEE Transactions on Network and Service Management, vol. 17, no. 2, pp. 892-905, 2020.
- [5] Lee, S., Kim, J., and Park, T., "Real-time Performance Monitoring Systems for IoT Device Management," Journal of Network and Computer Applications, vol. 145, article 102401, 2019.
- [6] OpenVSwitch Documentation, "OpenVSwitch Manual," Available: <https://www.openvswitch.org/support/dist-docs/>, 2023.
- [7] RYU SDN Framework Documentation, "RYU Controller Framework," Available: <https://ryu-sdn.org/documentation.html>, 2023.
- [8] Spring Boot Reference Documentation, "Spring Boot Features," Available: <https://spring.io/projects/spring-boot>, 2023.
- [9] React.js Documentation, "React - A JavaScript Library for Building User Interfaces," Available: <https://reactjs.org/docs/getting-started.html>, 2023.
- [10] ESP32 Technical Reference Manual, "ESP32 Series Datasheet," Espressif Systems, 2023.

8. APPENDICES

Appendix A: API Documentation

Backend API Endpoints:

Device Management APIs:

- POST /api/v1/device - Create new device
- GET /api/v1/device - Retrieve all devices
- GET /api/v1/device/{id} - Retrieve specific device
- PUT /api/v1/device/{id} - Update device information
- DELETE /api/v1/device/{id} - Delete device

Firmware Management APIs:

- POST /api/v1/device/{id}/firmware - Upload firmware to device
- GET /api/v1/device/{id}/metrics - Retrieve device performance metrics

Authentication APIs:

- POST /api/v1/auth - Authenticate user credentials

SDN Controller API Endpoints:

Flow Management APIs:

- POST /flow/add - Add new flow rule
- POST /flow/delete - Remove existing flow rule

Network Mode APIs:

- POST /flood/enable - Enable traditional networking mode
- POST /flood/disable - Enable SDN mode

IoT Device API Endpoints:

Device Control APIs:

- GET /on - Turn on LED indicator
- GET /off - Turn off LED indicator
- GET /blink - Activate LED blinking mode

Firmware Update APIs:

- GET /update - Access OTA update interface
- POST /update - Perform firmware update operation

9. REQUIREMENT ANALYSIS AND SPECIFICATION

Analysis and Purpose

The system is designed to address the critical need for secure and efficient firmware management in IoT environments. The analysis identified key requirements for centralized device management, secure firmware delivery, real-time performance monitoring, and dynamic network control capabilities.

Scope

The requirement scope encompasses all aspects of IoT device lifecycle management within an SDN-controlled network environment, including device registration, authentication, firmware updates, performance monitoring, and network security management.

Functional Requirements

FR1: The system shall provide complete CRUD operations for IoT device management including registration, modification, and removal of devices.

FR2: The system shall support secure firmware upload and deployment to registered IoT devices through SDN-controlled network channels.

FR3: The system shall collect and display real-time performance metrics from IoT devices including memory usage, CPU frequency, temperature, and connectivity status.

FR4: The system shall provide dynamic network mode switching between SDN-controlled and traditional networking modes.

FR5: The system shall implement user authentication for system access control.

FR6: The system shall provide a web-based user interface for all system management functions.

Non-functional Requirements

NFR1: The system shall support response times of less than 2 seconds for user interface operations and less than 30 seconds for firmware update initiation.

NFR2: The system shall maintain 99% uptime during normal operating conditions with appropriate error handling and recovery mechanisms.

NFR3: The system shall support secure communication protocols and implement proper access controls to prevent unauthorized system access.

NFR4: The system shall provide comprehensive logging and monitoring capabilities for system administration and troubleshooting.

Software Requirements

- Frontend: React.js 19.0.0 with supporting libraries for UI components and HTTP communications
- Backend: Spring Boot 3.4.3 with Spring Data JPA
- Database: MySQL 8.0 for reliable data persistence
- SDN Controller: RYU framework 4.34 for OpenFlow and REST API support

Hardware Requirements

- Development Server: Multi-core processor with minimum 8GB RAM and 100GB storage
- Network Infrastructure: OpenVSwitch-capable hardware or virtual environment
- IoT Devices: ESP32 microcontrollers with Wi-Fi connectivity
- Network Connectivity: Ethernet interfaces for SDN switch connectivity

Environmental Conditions

The system requires stable network connectivity, appropriate power supply for all components, and suitable physical environment for hardware components including proper ventilation and temperature control.

Safety and Security Requirements

The system implements secure communication channels through SDN flow control, user authentication, comprehensive audit logging, and proper error handling to prevent system compromise or data loss.

Software Quality Attributes

Performance: The system maintains user interfaces and efficient firmware delivery with acceptable resource utilization across all components.

Reliability: Comprehensive error handling and recovery mechanisms ensure consistent system operation even under adverse conditions.

Usability: The web-based interface provides intuitive navigation and clear feedback for all user interactions.

Maintainability: Modular architecture and comprehensive documentation facilitate system maintenance and future enhancements.

